

SII Prueba Técnica – Laravel 12

Proyecto: Sistema Integral de Información (SII) – Versión Prueba Técnica

Framework: Laravel 12

PHP: 8.2+

Base de datos: SQLite (por defecto) o MySQL

Duración estimada: 6–8 horas (máximo 1 día)

Instalación rápida

```
# 1. Clonar o copiar el proyecto
cd prueba-tecnica-unisant

# 2. Instalar dependencias
composer install

# 3. Configurar entorno
cp .env.example .env
php artisan key:generate

# 4. Base de datos (SQLite ya está configurada)
touch database/database.sqlite
php artisan migrate --seed

# 5. Iniciar servidor
php artisan serve

> **¿`php artisan serve` no funciona?** En Windows con PHP 8.4+ puede fallar. Usa alguna de
estas alternativas:
> - **Herd:** Si tienes Laravel Herd instalado, el proyecto debería aparecer automáticamente
en `http://prueba-tecnica-unisant.test`.
> - **Servidor PHP nativo:** `php -S 127.0.0.1:8000 -t public`
> - **Otra herramienta:** XAMPP, WAMP, Docker, etc.
```

Credenciales de prueba:

- **Admin:** admin@unisant.test / password
- **Sede:** sede@unisant.test / password
- **Control Escolar:** ce@unisant.test / password

Estructura del proyecto

```
app/
  Models/           → Alumno, Programa, Materia, Pago, Sede, Inscripcion, Adeudo, User
  Http/Controllers/ → AlumnoController, ProgramaController, PagoController,
DashboardController, ReporteController, Api/AlumnoApiController
  Http/Middleware/   → NivelMiddleware
  Jobs/              → GenerarReporteDeudasJob
```

```
database/
  migrations/          → 10 migraciones con datos de prueba
  seeders/             → UserSeeder, SedeSeeder, ProgramaSeeder, AlumnoSeeder
resources/views/
  layouts/app.blade.php
  dashboard/index.blade.php
  alumnos/index.blade.php, create.blade.php
  programas/index.blade.php
  pagos/create.blade.php, index.blade.php
  auth/login.blade.php, register.blade.php
routes/
  web.php              → Rutas principales
  api.php              → Endpoints API
  auth.php             → Login / Register / Logout
```

⚠ Estado del proyecto

Este proyecto NO está completo. Fue construido sobre una base real que ha sufrido refactorizaciones, cambios de equipo y prisas de entrega. Como suele pasar en sistemas legacy, el código funciona... hasta que no.

Te sugerimos explorar con ojo crítico estas áreas:

- **Base de datos:** Revisa los tipos de dato elegidos en las migraciones y cómo se comportan los modelos al serializar ciertos campos.
 - **Backend:** Algunos listados pueden volverse lentos con más registros. Hay operaciones que quizá deberían ser atómicas.
 - **Seguridad:** No todo endpoint responde como esperarías cuando no hay sesión activa. Algunas comparaciones podrían ser más estrictas.
 - **Frontend:** Interactúa con los formularios y tablas como si fueras usuario final. Nota si algo se siente frágil.
 - **Procesos en segundo plano:** El sistema genera reportes automáticos. Pregúntate qué pasa si algo falla en mitad del proceso.
 - **Consistencia de datos:** Los seeders insertan información realista, pero no toda es "limpia". ¿Qué tan robusto es el sistema ante datos imperfectos?
-

📖 Documentación adicional

- docs/INSTRUCCIONES_CANDIDATO.md – Guía paso a paso para el candidato.
 - docs/PRUEBA_TECNICA.md – Tareas específicas, tiempos estimados y entregables.
 - docs/EVALUACION.md – Rúbrica y criterios de evaluación (para el reclutador).
-

📁 Entrega

Al finalizar la prueba, el candidato debe:

1. Crear un **repositorio propio** (GitHub, GitLab, Bitbucket, etc.) con el código corregido.
 2. Incluir el archivo **RESPUESTA.md** en la raíz.
 3. Enviar el **enlace al repositorio** al correo: `ltepepa@unisant.edu.mx`
-

📄 Licencia

Este proyecto es de uso exclusivo para pruebas técnicas de reclutamiento. No debe usarse en producción.